

Your Paper Title Here

Paper #XXX

October 5, 2026

Abstract:

We present SystemName, a [describe system] that [key capability]. [Problem statement: why existing approaches fall short.] SystemName addresses this through [key technique/insight]. We evaluate SystemName on [workloads/benchmarks] and show that it achieves [X]% improvement in [metric] over [baseline], while reducing [other metric] by [Y]×.

1 Introduction

Modern systems face the challenge of [describe problem] [2]. As workloads grow in scale and complexity, existing approaches such as [prior work] [1] struggle to [limitation].

Key Insight. Our key observation is that [insight]. This enables SystemName to [capability] without [drawback of prior approaches].

We make the following contributions:

- We identify [problem/opportunity] and characterize its impact on [workloads] (Section 2).
- We design SystemName, which introduces [technique] to address [challenge] (Section 3).
- We implement SystemName in [X] lines of [language] and integrate it with [existing system] (Section 4).
- We evaluate SystemName on [benchmarks] and demonstrate [X]× improvement over [baseline] (Section 5).

2 Background and Motivation

2.1 Problem Context

Describe the system context and relevant background. Prior work [4, 7] has explored [related area], but [gap remains].

2.2 Motivating Example

Figure 1 shows [measurement] across [workloads]. We observe that [finding], which motivates our approach.

Motivating measurement or characterization study

Figure 1: [Description of motivating measurement.] We observe that [key finding] across [N] production workloads, motivating the need for [your approach].

Algorithm 1 Core algorithm of SystemName

```
1: Input: workload  $W$ , resources  $R$ 
2: Output: scheduling plan  $P$ 
3: Initialize plan  $P \leftarrow \emptyset$ 
4: for each task  $t_i \in W$  do
5:   Estimate resource demand  $d_i \leftarrow \text{Predict}(t_i)$ 
6:   if Available( $R$ )  $\geq d_i$  then
7:      $P \leftarrow P \cup \{(t_i, \text{Allocate}(R, d_i))\}$ 
8:   else
9:     Enqueue  $t_i$  for deferred scheduling
10:  end if
11: end for
12: return  $P$ 
```

3 Design

Figure 2 shows the overall architecture of SystemName. The system consists of [N] key components: [list].

3.1 Component A: [Name]

Describe the first key component. A formal specification of the core algorithm can be found in Algorithm 1.

3.2 Component B: [Name]

Describe the second key component. The expected throughput can be modeled as:

$$T = \frac{N \cdot B}{L + \frac{B}{C}} \quad (1)$$

System architecture diagram showing key components and data flow

Figure 2: Architecture of SystemName. [Component A] handles [function], while [Component B] manages [function]. Arrows indicate [data/control flow].

where N is the number of parallel workers, B is the batch size, L is the network latency, and C is the per-worker compute rate.

3.3 Handling Edge Cases

Discuss how the design handles failures, stragglers, or other edge cases important in production systems.

4 Implementation

We implement SystemName in approximately [X]K lines of [language]. Key implementation details include:

Threading Model. [Describe the threading/concurrency model.]

Integration. SystemName integrates with [existing system] by [method of integration]. We modify [N] lines of the original codebase.

5 Evaluation

We evaluate SystemName to answer the following questions:

1. How does SystemName compare to state-of-the-art systems?
2. What is the contribution of each design component?
3. How does SystemName scale with increasing workload?
4. What overhead does SystemName introduce?

5.1 Experimental Setup

Testbed. We run experiments on a cluster of [N] machines, each with [CPU model], [X] GB RAM, and [GPU model if applicable], connected via [network].

Workloads. We use [N] workloads from [source]: [list workloads]. Table 1 summarizes their characteristics.

Baselines. We compare against [N] baselines: (1) [Baseline A] [6], (2) [Baseline B] [5], and (3) [Baseline C].

Table 1: Workload characteristics used in evaluation. [Describe what the columns represent.]

Workload	Tasks	Data (GB)	Duration	Type
WorkloadA	1,024	128	2.4 h	Batch
WorkloadB	512	64	1.1 h	Streaming
WorkloadC	4,096	512	8.7 h	ML Train
WorkloadD	256	32	0.5 h	Serving

Table 2: End-to-end performance comparison. Bold indicates best result. SystemName achieves the highest throughput and lowest p99 latency across all workloads.

System	Throughput (Kops/s)	p50 Latency (ms)	p99 Latency (ms)
Baseline A	125.3	4.2	18.7
Baseline B	142.1	3.8	15.2
Baseline C	98.6	5.1	22.4
SystemName	187.4	2.9	9.8

5.2 End-to-End Performance

Table 2 shows the end-to-end performance comparison. SystemName achieves [X]% higher throughput and [Y]% lower latency compared to [best baseline].

5.3 Ablation Study

To understand the contribution of each component, we evaluate variants of SystemName with individual components disabled. Figure 3 shows the results.

5.4 Scalability

We evaluate how SystemName scales from [N] to [M] nodes. As shown in Figure 4, SystemName achieves near-linear scaling up to [K] nodes.

6 Discussion

Limitations. [Discuss known limitations of your system.]

Bar chart: SystemName vs. variants with components disabled

Figure 3: Ablation study results. Each bar represents SystemName with one component removed. Component A contributes [X]% and Component B contributes [Y]% of the total improvement.

Line chart: throughput vs. number of nodes for each system

Figure 4: Scalability comparison. SystemName achieves [X]% of ideal linear scaling at [K] nodes, compared to [Y]% for [baseline].

Lessons Learned. [Share insights from building and deploying the system.]

7 Related Work

[Category A] Systems. Prior work on [category] [2, 1] focuses on [aspect]. SystemName differs by [distinction].

[Category B] Approaches. [Other approaches] [3, 6] address [problem] through [method]. In contrast, SystemName [distinction].

8 Conclusion

We presented SystemName, a [description] that [key capability]. Through [technique], SystemName achieves [improvement] over state-of-the-art systems. Our evaluation on [workloads] demonstrates [key results]. [Optional: future work direction.]

References

- [1] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., ET AL. TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (Savannah, GA, 2016), USENIX Association, pp. 265–283.
- [2] DEAN, J., AND GHEMAWAT, S. MapReduce: Simplified data processing on large clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (San Francisco, CA, 2004), USENIX Association, pp. 137–150.
- [3] LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21, 7 (1978), 558–565.
- [4] MORITZ, P., NISHIHARA, R., WANG, S., TUMANOV, A., LIAW, R., LIANG, E., ELIBOL, M., YANG, Z., PAUL, W., JORDAN, M. I., AND STOICA, I. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (Carlsbad, CA, 2018), USENIX Association, pp. 561–577.
- [5] ONGARO, D., AND OUSTERHOUT, J. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)* (Philadelphia, PA, 2014), USENIX Association, pp. 305–319.
- [6] VERMA, A., PEDROSA, L., KORUPOLU, M., OPPENHEIMER, D., TUNE, E., AND WILKES, J. Large-scale cluster management at Google with Borg. In *Proceedings of the 10th European Conference on Computer Systems (EuroSys)* (Bordeaux, France, 2015), ACM, pp. 1–17.
- [7] ZAHARIA, M., CHOWDHURY, M., DAS, T., DAVE, A., MA, J., MCCAULEY, M., FRANKLIN, M. J., SHENKER, S., AND STOICA, I. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (San Jose, CA, 2012), USENIX Association, pp. 15–28.